React Native Akadémia

Lektori



Jozef Pavlov



Lukáš Mikolaj



Michal Géci



Dominik Štefan



Ivan Krzák

Prehľad lekcií

1. Úvod do RN Akadémie
2. Základy Reactu
3. Základy React Native
4. Debugging
5. Navigácia
6. Asynchrónne procesy
7. Globálny state management
8. Spracovanie formulárov
9. Využitie natívnych funkcionalít
10. Unit Testing
11. Advanced UI a Storybook

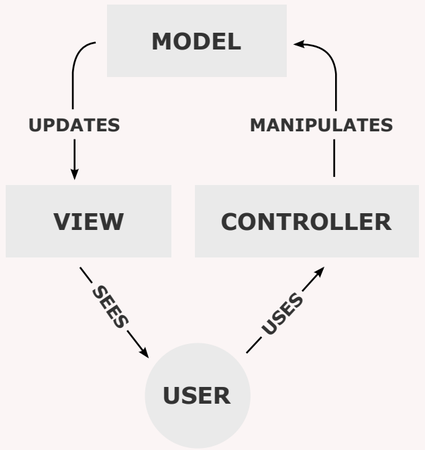
React ako základ

- Open-source JavaScript knižnica
- Založená na budovaní rozhrania pomocou komponentov
- Počiatky Reactu sa datujú do rokov 2011 až 2013

Na ktorú funkcionalitu alebo aplikáciu bol React nasadený ako prvú?

1. Facebook Groups
2. Instagram
3. Facebook News Feed
4. Facebook Ads Manager

Model-View-Controller (MVC)



Separation of Concerns

- Model - dáta a ich stav
- Controller - manipulácia dát
- View - zobrazuje aktualizované dáta

Todo App: HTML šablóna

```
<!DOCTYPE html>
<html lang="en">

<head>
  <title>Todo App</title>
</head>

<body>
  <div id="root"></div>

  <script src="script.js"></script>
</body>

</html>
```

Todo App: Model

```
class Model {
  constructor() {
    this.todos = JSON.parse(localStorage.getItem('todos')) || []
  }

  bindTodoListChanged(callback) {
    this.onTodoListChanged = callback
  }

  _commit(todos) {
    this.onTodoListChanged(todos)
    localStorage.setItem('todos', JSON.stringify(todos))
  }

  addTodo(todoText) {
    const todo = {
      id: this.todos.length > 0 ? this.todos[this.todos.length - 1].id + 1 : 1,
      text: todoText,
    }

    this.todos.push(todo)
    this._commit(this.todos)
  }
```

Todo App: View

```
class View {
  constructor() {
    this.app = this.getElement('#root')
    this.submitButton = this.createElement('button')
    this.submitButton.textContent = 'Add Todo'
    this.todoList = this.createElement('ul', 'todo-list')
    this.app.append(this.submitButton, this.todoList)
  }

  createElement(tag, className) {
    const element = document.createElement(tag)
    if (className) element.classList.add(className)
    return element
  }

  getElement(selector) {
    const element = document.querySelector(selector)
    return element
  }

  displayTodos(todos) {
    todos.forEach(todo => {
      const li = this.createElement('li')
```

Todo App: Controller

```
class Controller {
  constructor(model, view) {
    this.model = model
    this.view = view

    this.model.bindTodoListChanged(this.onTodoListChanged)
    this.view.bindAddTodo(this.handleAddTodo)
  }

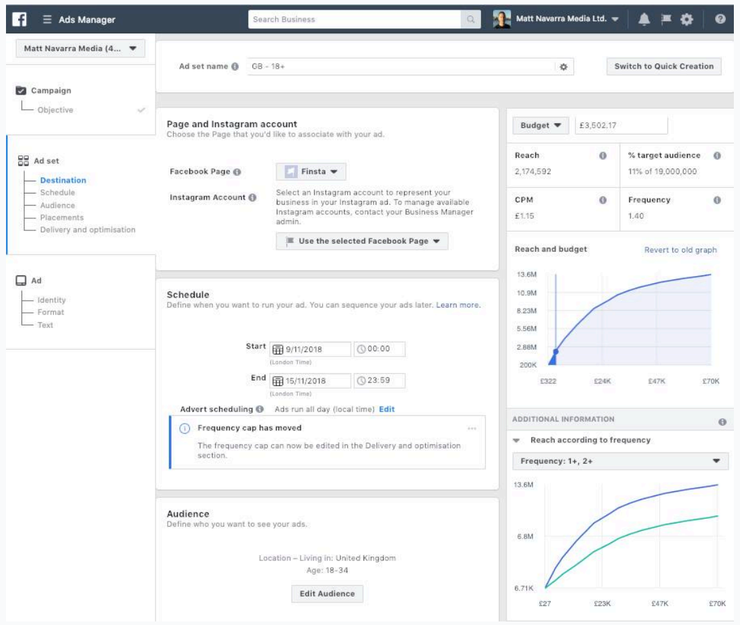
  onTodoListChanged = todos => {
    this.view.displayTodos(todos)
  }

  handleAddTodo = todoText => {
    this.model.addTodo(todoText)
  }
}
```

Fungovanie Todo App

1. Pomocou HTML šablóny a MVC komponentov sa v `div` elemente s `id='root'` vykreslí tlačidlo s názvom `Add Todo`.
2. Používateľ klikne na `Add Todo` tlačidlo, čo odchytí `click` event listener **View** komponentu a odošle túto informáciu **Controller** komponentu.
3. **Controller** následne povie **Model** komponentu, aby vytvoril novú Todo položku a pridal ju do zoznamu Todos.
4. **Model** upraví zoznam Todos s novým itemom, uloží aktualizovaný zoznam do lokálneho úložiska a posunie ho naspäť **Controller** komponentu.
5. **Controller** tento aktualizovaný zoznam Todos predá **View** komponentu, ktorý ho zobrazí pomocou HTML šablóny.
6. Používateľ vidí zoznam Todos s novou `New Item` todo položkou.

Facebook Ads Manager & React



Ads Manager: bod, kedy pridanie akejkoľvek zmeny bolo extrémne náročné až nemožné

Ako tieto problémy React rieši?

- Budovanie UI pomocou komponentov
- Na rozdiel od HTML šablón sa na Views používa reálny JavaScript so všetkými funkcionalitami
- Zjednotenie markupu (JSX) a logiky Views zabezpečuje jednoduché rozširovanie a udržiavanie
- Deklaratívny štýl manipulácie UI namiesto imperatívnych paradigiem

Aký bol pôvodný názov Reactu?

1. Bolt
2. FBolt
3. FaxJS
4. XHP



TIME FOR

localhost.company

HYDRATION BREAK

Imperatívne vs deklaratívne UI programovanie

Syntax

- **Imperatívny prístup:** programovanie krok po kroku, inštrukcie ako sa má niečo vykonať.
- **Deklaratívne programovanie:** zameranie na výsledok ako na spôsob dosiahnutia výsledku.

Manažment stavu

- **Imperatívne programovanie:** explicitný manuálny manažment stavov.
- **Deklaratívny prístup:** zadefinovanie stavu, manažment je na frameworku.

Jednoduchý formulár, kde používateľ môže zadať odpoveď

1. Začnem písať do formuláru text, `Submit` tlačidlo sa **sprístupní**.
2. Stlačím `Submit` tlačidlo, formulár a tlačidlo sa **znesprístupnia** a zobrazí sa **načítavanie**.
3. Ak sa request skončí úspešne, formulár sa **skryje** a zobrazí sa **úspešná hláška**.
4. Ak sa request skončí chybou, formulár sa **sprístupní** a zobrazí sa **chybová hláška**.

Imperatívny formulár: HTML

```html
<form id="form">
  <h2>City quiz</h2>
  <p>
    What city is located on two continents?
  </p>
  <textarea id="textarea"></textarea>
  <br />
  <button id="button" disabled>Submit</button>
  <p id="loading" style="display: none">Loading...</p>
  <p id="error" style="display: none; color: red;"></p>
</form>
<h1 id="success" style="display: none">That's right!</h1>

<style>
* { box-sizing: border-box; }
body { font-family: sans-serif; margin: 20px; padding: 0; }
</style>
```

Interakcia formulára

1. Začnem písať do formuláru text, `Submit` tlačidlo sa **sprístupní**.
2. Stlačím `Submit` tlačidlo, formulár a tlačidlo sa **znesprístupnia** a zobrazí sa **načítavanie**.
3. Ak sa request skončí úspešne, formulár sa **skryje** a zobrazí sa **úspešná hláška**.
4. Ak sa request skončí chybou, formulár sa **sprístupní** a zobrazí sa **chybová hláška**.

Imperatívny formulár: JavaScript

```
async function handleFormSubmit(e) {
  e.preventDefault();
  disable(textarea);
  disable(button);
  show(loadingMessage);
  hide(errorMessage);
  try {
    await submitForm(textarea.value);
    show(successMessage);
    hide(form);
  } catch (err) {
    show(errorMessage);
    errorMessage.textContent = err.message;
  } finally {
    hide(loadingMessage);
    enable(textarea);
    enable(button);
  }
}

function handleTextareaChange() {
  if (textarea.value.length === 0) {
    disable(button);
```

Interakcia formulára

1. Začnem písať do formuláru text, `Submit` tlačidlo sa **sprístupní**.
2. Stlačím `Submit` tlačidlo, formulár a tlačidlo sa **znesprístupnia** a zobrazí sa **načítavanie**.
3. Ak sa request skončí úspešne, formulár sa **skryje** a zobrazí sa **úspešná hláška**.
4. Ak sa request skončí chybou, formulár sa **sprístupní** a zobrazí sa **chybová hláška**.

React a deklaratívne UI

Princíp

- V Reacte nemanipulujeme UI priamo – **nepristupujeme ku komponentom priamo**, aby sme ich skryli či sprístupnili.
- Namiesto toho **deklarujeme**, čo chceme zobraziť, a React si sám zistí, ako má upraviť UI.

5 jednoduchých krokov

1. Identifikujme, aké rôzne vizuálne stavy môže mať náš komponent.
2. Určme, čo spúšťa zmeny jeho stavov.
3. Vyjadrime jeho stav pomocou `useState`.
4. Odstráňme všetky nadbytočné state premenné.
5. Napojme event handlery pre nastavovanie stavu.

1. Identifikácia vizuálnych stavov

Interakcia formulára

1. Začnem písať do formuláru text, `Submit` tlačidlo sa **sprístupní**.
2. Stlačím `Submit` tlačidlo, formulár a tlačidlo sa **znesprístupnia** a zobrazí sa **načítavanie**.
3. Ak sa request skončí úspešne, formulár sa **skryje** a zobrazí sa **úspešná hláška**.
4. Ak sa request skončí chybou, formulár sa **sprístupní** a zobrazí sa **chybová hláška**.

Vizuálne stavy

- **Prázdny (empty):** žiaden text, neprístupné tlačidlo.
- **Písanie (typing):** zadaný text, sprístupnené tlačidlo.
- **Potvrdenie (submitting):** celý formulár neprístupný, zobrazené načítavanie.
- **Úspech (success):** formulár skrytý, zobrazená úspešná hláška.
- **Chyba (error):** formulár sprístupnený, zobrazená chybová hláška.

```
export default function Form({
  // options 'empty', 'typing', 'submitting', 'error', 'success':
  status = 'empty'
}) {
  if (status === 'success') {
    return <h1>That's right!</h1>
  }
  return (
    <>
      <h2>City quiz</h2>
      <p>
        In which city is there a billboard that turns air into drinkable wa
      </p>
      <form>
        <textarea disabled={
          status === 'submitting'
        } />
        <br />
        <button disabled={
          status === 'empty' ||
          status === 'submitting'
        }>
          Submit
```

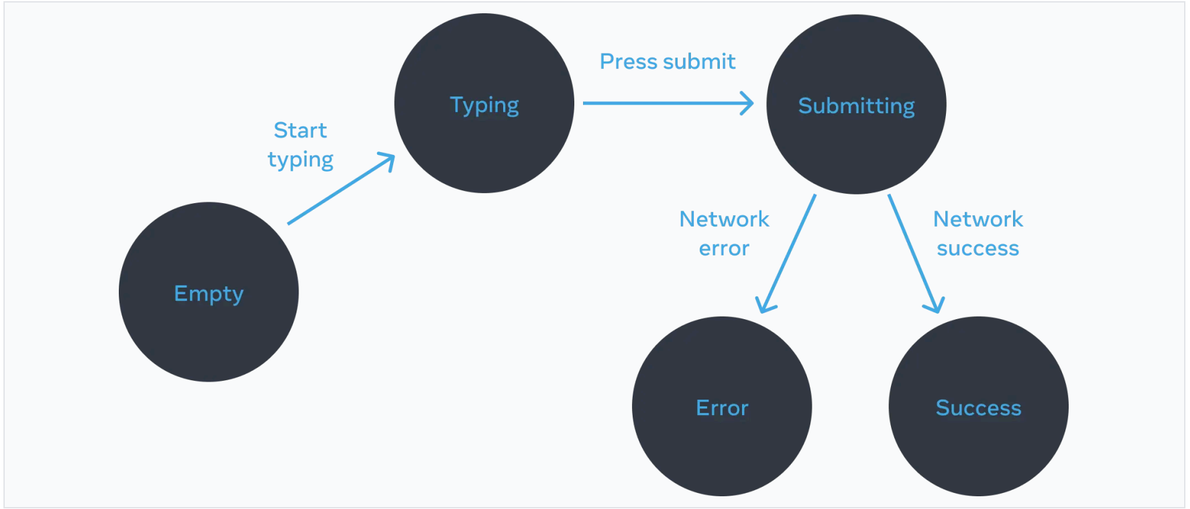
2. Určenie spúšťačov zmien stavov

Spúšťače zmien

- **vstupy používateľa** - stlačenie tlačidla, vloženie textu, navigácia cez link...
- **vstupy počítača** - ukončenie asynchrónnej operácie ako odpoveď zo servera, načítanie obrázku a pod.

Zmeny stavov

- **Zmena textového vstupu** (používateľ) zmení stav z `empty` na `typing`, alebo naopak, podľa toho, či sa v textovom poli nachádza nejaký text.
- **Kliknutie na `Submit` tlačidlo** (používateľ) vyvolá zmenu stavu na `submitting`.
- **Úspešná odpoveď zo servera** (počítač) zmení stav na `success`.
- **Chybová odpoveď zo servera** (počítač) zmení stav na `error`.



3. Vyjadrenie stavov pomocou useState

- reprezentácia stavov v pamäti – `useState` hook
- **jednoduchosť** je kľúčová
- nevyhnutné stavy

Nevyhnutné stavy:

- `answer` – text odpovede používateľa
- `error` – chyba, ak odpoveď nie je správna

- `isEmpty` – pole je prázdne
- `isTyping` – používateľ práve píše
- `isSubmitting` – formulár sa odosiela
- `isSuccess` – odpoveď bola správna
- `isError` – odpoveď bola nesprávna

```
const [answer, setAnswer] = useState('');
const [error, setError] = useState(null);

const [isEmpty, setIsEmpty] = useState(true);
const [isTyping, setIsTyping] = useState(false);
const [isSubmitting, setIsSubmitting] = useState(false);
const [isSuccess, setIsSuccess] = useState(false);
const [isError, setIsError] = useState(false);
```

4. Odstránenie nadbytočných stavov

- odstránenie duplicít
- zabránenie nevalidným stavom
- otázky pri refaktoringu:
  - **Spôsobuje tento stav paradox?** - niektoré zo stavových premenných nemôžu mať v jednom čase rovnakú hodnotu
  - **Je dostupný z inej premennej?** - viacero premenných vyjadrujúcich rovnaký stav
  - **Vieme získať rovnakú informáciu z opačnej hodnoty inej premennej?**

isTyping	isSubmitting	status
true	false	typing
false	true	submitting
false	false	success
true	true	paradox

Pôvodný stav

```
const [answer, setAnswer] = useState('');
const [error, setError] = useState(null);

const [isEmpty, setIsEmpty] = useState(true);
const [isTyping, setIsTyping] = useState(false);
const [isSubmitting, setIsSubmitting] = useState(false);
const [isSuccess, setIsSuccess] = useState(false);
const [isError, setIsError] = useState(false);
```

Refaktorovaný stav

```
const [answer, setAnswer] = useState('');
const [error, setError] = useState(null);
// 'typing', 'submitting', or 'success'
const [status, setStatus] = useState('typing');
```

5. Napojenie event handlerov

```
function submitForm(answer) {
  // Pretend it's hitting the network.
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      let shouldError = answer.toLowerCase() !== 'lima';
      if (shouldError) {
        reject(new Error('Good guess but a wrong answer. Try again!'));
      } else {
        resolve();
      }
    }, 1500);
  });
}
```

```
async function handleSubmit(e) {
  e.preventDefault();
  setStatus('submitting');
  try {
    await submitForm(answer);
    setStatus('success');
  } catch (err) {
    setStatus('typing');
    setError(err);
  }
}
```

```
function handleTextareaChange(e) {
  setAnswer(e.target.value);
}
```

```
import { useState } from 'react';

export default function Form() {
  const [answer, setAnswer] = useState('');
  const [error, setError] = useState(null);
  const [status, setStatus] = useState('typing');

  if (status === 'success') {
    return <h1>That's right!</h1>;
  }

  async function handleSubmit(e) { ... }

  function handleTextareaChange(e) { ... }

  return (
    <>
      <h2>City quiz</h2>
      <p>
        In which city is there a billboard that turns air into drinkable wa
      </p>
      <form onSubmit={handleSubmit}>
```



TIME FOR

localhost.company

HYDRATION BREAK

Tvorba UI pomocou JSX

Čo je JSX?

- Rozšírenie syntaxe pre JavaScript, ktoré umožňuje tvorbu UI v štruktúre podobnej HTML markupu, ale v JavaScript súboroch.
- React drží renderovaciu logiku a UI markup na jednom mieste – v **komponentoch**.
- JSX sa podobá na HTML, no má o niečo striktnejšie pravidlá a dokáže zobrazovať dynamický obsah.

```
function Form() {
  // logic
  onClick() {...}
  onSubmit() {...}

  // UI
  return (
    <form onSubmit>
      <input onClick />
      <input onClick />
    </form>
  );
}
```

Základné pravidlá JSX

Jeden root element

- JSX komponent musí vrátiť práve jeden root element
- Ak potrebujeme vrátiť viacero elementov, musia byť wrapnuté jedným rodičovským elementom.
- Ak nepotrebujeme použiť žiaden konkrétny element, môžeme využiť Fragment `<> </>`

```
<>
  <h1>localhost.company</h1>
  <img
    src="https://bit.ly/4cdrOew"
    alt="localhost.company"
    class="photo"
  >
  <ul>
    ...
  </ul>
</>
```

Všetky tagy sú uzatvorené

- JSX vyžaduje, aby boli všetky tagy uzatvorené
- Samouzatváracie tagy: `<img />`
- Wrapovacie tagy: `<li>IT's Safe with Us</li>`

```
<>
  <img
    src="https://cdn.prod.website-files.com/6702f
    alt="localhost.company"
    class="photo"
  /> // self-closing tag
  <ul>
    <li>IT's Easy with Us</li>
    <li>IT's Safe with Us</li>
    <li>IT's Swift with Us</li>
    <li>IT's Epic with Us</li>
  </ul> // wrapping tag
</>
```

camelCase takmer pre všetko

- JSX sa v konečnom dôsledku stane JavaScript-om, takže aj atribúty písané v JSX sa stanú kľúčmi JavaScript objektov
- JavaScript má určité limitácie pre názvy premenných
- väčšina HTML atribútov sa píše camelCase
  - `strokeWidth` namiesto `stroke-width`
  - `className` namiesto `class`

```
<img
  src="https://cdn.prod.website-files.com/6702f5f
  alt="localhost.company"
  className="photo"
/>
```

JSX pod kapotou

- rozšírenie syntaxe pre JavaScript
- transpilácia na JavaScript
- každý JSX element je JavaScript funkcia

JSX kód:

```jsx
import React from 'react';

function App() {
  return <div id="root">IT's Equal with Us</div>;
}
```

JavaScript reprezentácia:

```jsx
import React from 'react';

function App() {
  return React.createElement('div', { id: 'root' }, "IT's Equal with Us");
}
```

Disclaimer: `React.createElement` je Legacy API a od React 17 je možné používať nový spôsob transformácie JSX (the new JSX transform), ktorý je zároveň od React 19 povinný. `React.createElement` API je ale stále dostupné a momentálne nie je v pláne ho odstrániť.

new JSX transform:

```jsx
function App() {
  return <div id="root">IT's Equal with Us</div>;
}
```

Po skompilovaní:

```jsx
// Inserted by a compiler (don't import it yourself!)
import {jsx as _jsx} from 'react/jsx-runtime';

function App() {
  return _jsx('div', {
    id: 'root',
    children: "IT's Equal with Us"
  });
}
```

Kto je zodpovedný za túto transformáciu JSX na JavaScript?

1. Hermes
2. Babel
3. TypeScript
4. Fabric

React.createElement

Výstup volaní

- `<Greeting name="localhost" />`
- `React.createElement('Greeting', { name: 'localhost'})`

je jednoduchý popis časti používateľského rozhrania.

Zjednodušený popis časti rozhrania (element):

```
{
  type: Greeting,
  props: {
    name: 'localhost'
  },
  key: null,
  ref: null,
}
```

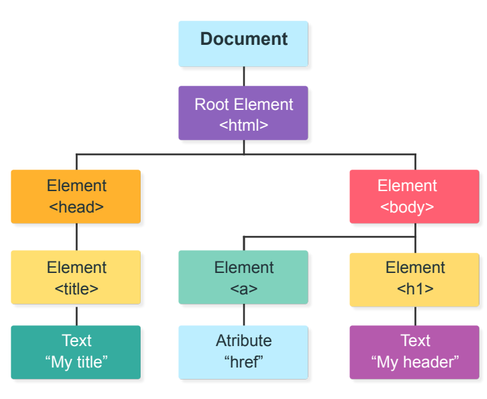
Dôležité

- Vytvorenie tohto objektu **nespôsobuje vykreslenie komponentu a ani nevytvára žiadne DOM elementy**.
- Je to len popis, alebo akási šablóna, na základe ktorej React neskôr vyrenderuje komponent.
- Vytváranie elementov je extrémne ľahké, renderovanie je náročné.

DOM a Virtuálny DOM

DOM (Document Object Model)

- Stromová štruktúra vytvorená prehliadačom z HTML kódu.
- Priama manipulácia s reálnym DOM-om je výpočtovo pomalá a prácna.
- JavaScript komunikuje prostredníctvom DOM API, napr. `document.getElementById('root')`.



Virtuálny DOM (VDOM)

- Programátorský koncept: „virtuálne“ UI sa udržiava v pamäti a synchronizuje s „reálnym“ DOM-om pomocou knižnice (napr. ReactDOM).
- Tento proces sa nazýva **rekonciliácia (reconciliation)**.
- Umožňuje deklaratívne API: Reactu poviete želaný stav a on sa postará o DOM.

Rekonciliácia verzus renderovanie

Rekonciliátor (reconciler)

- Vygeneruje strom uzlov v pamäti.
- Pri aktualizácii (zvyčajne cez `setState`) vygeneruje nový strom.
- Porovná rozdiely (diffing) oproti predchádzajúcemu stromu.
- Poskytovaný jadrom Reactu (React core).

Renderer

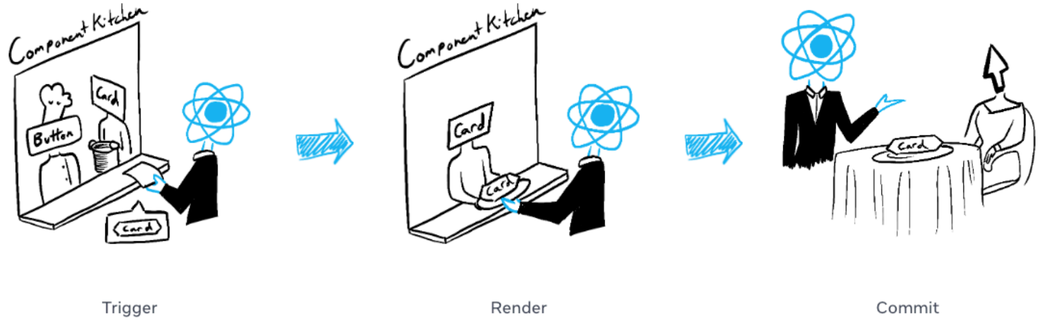
- Použije informácie z rekonciliácie na samotnú aktualizáciu aplikácie.
- **React DOM:** renderovanie pre webový DOM.
- **React Native:** natívne zobrazenia pre iOS a Android (zdieľajú rovnaký rekonciliátor, no vlastný renderer).

Aký renderer sa používa pre React Native?

1. Fiber
2. Fabric
3. Hermes
4. ReactDOM

Render & Commit

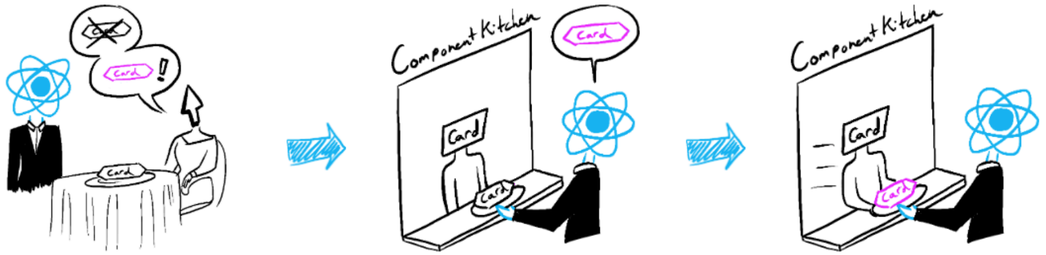
1. **Triggering** a render (odovzdanie objednávky od zákazníka kuchárom).
2. **Rendering** the component (príprava objednávky v kuchyni).
3. **Committing** to the DOM (prinesenie objednávky zákazníkovi na stôl).



Trigger Render Commit

1. Triggering a render

- Inicializačný render alebo zmena stavu pomocou `setState`.
- „Odovzdanie objednávky od zákazníka kuchárom."



State update... ...triggers... ...render!

Render & Commit

2. Rendering

- React zavolá komponent, proces je rekurzívny.
- „Príprava objednávky v kuchyni."

- Pri **inicializačnom renderi** React zavolá root komponent.
- Pri **následných re-renderoch** React volá komponent, ktorého zmena stavu spustila render.

Gallery.js:

```
// Gallery.js
export default function Gallery() {
  return (
    <section>
      <h1>Inspiring Sculptures</h1>
      <Image />
      <Image />
      <Image />
    </section>
  );
}

function Image() {
  return (
    <img
      src="https://react.dev/images/docs/scientists/ZF6s192.jpg"
      alt="'Floralis Genérica' by Eduardo Catalano: a gigantic metallic flo
    />
  );
}
```

index.js:

```
// index.js
import Gallery from './Gallery.js';
import { createRoot } from 'react-dom/client';

const root = createRoot(document.getElementById('root'))
root.render(<Gallery />);
```

Render & Commit

Renderovanie musí byť vždy čistý výpočet (pure calculation):

Rovnaký vstup, rovnaký výstup

- Pri poskytnutí rovnakého vstupu by mal komponent vždy vrátiť rovnaký JSX.
- Ak si niekto objedná šalát s paradajkami, nemal by nikdy dostať šalát s cibuľou.

Stará sa len o seba

- Renderovanie komponentu by nemalo ovplyvniť žiaden iný objekt alebo premennú vytvorenú pred renderom.
- Moja objednávka by nemala zmeniť objednávku niekoho iného.

!!! Nedodržiavanie týchto pravidiel vedie k mätúcim bugom a neočakávanému správaniu aplikácie.

React zapíše zmeny do DOM

3. Committing to DOM

- React zapíše zmeny do DOM-u.
- „Prinesenie objednávky zákazníkovi na stôl."

```
/// Clock.js

export default function Clock({ time }) {
  return (
    <>
      <h1>{time}</h1>
      <input />
    </>
  );
}
```

- Pri **inicializačnom renderi** React využíva `appendChild()` DOM API.
- Pri **následných re-renderoch** React aplikuje len nevyhnutné operácie (vyrátané počas renderovania).

Commit & Browser Painting

- **Inicializačný render:** React využíva `appendChild()` DOM API pre všetky uzly.
- **Re-render:** React aplikuje len nevyhnutné operácie (v `Clock` iba `<h1>` s novou `time` hodnotou, stav `<input>` ostáva nedotknutý).
- **Browser rendering (Painting):** Prehliadač prekreslí obrazovku po úprave DOM-u.